CS2109S
AY25/26 Sem 1
by ngmh

Introduction
- PEAS: Performance Measure, Environment, Actuators, Sensors
- A rational agent chooses actions that maximise performance measure
- Agent percepts with sensors and performs actions with actuators
- Environments:
  - Fully Observable (v/s Partially Observable): Agent's sensors give it access to complete state of environment, no need to keep track of state and no hidden information
  - Single agent (v/s Multi agent): Presence of multiple agents in environment
  - Deterministic (v/s Stochastic): Next state of environment is completely determined by current state and action executed by agent; if other rational agents actions matter it is also strategic, ignore uncertainty from actions of other agents even though agent cannot predict others actions
  - Deterministic: Given a state and set of actions, we can precisely determine the next state
  - Stochastic: Given a state and set of actions, we cannot determine the exact next state, but we know the probability of reaching each possible next state
  - Episodic (v/s Sequential): Experience is divided into atomic episodes of agent perceiving then performing an action, and decision making is only dependent on the current episode
  - Static (v/s Dynamic): Environment is unchanged while an agent is deliberating; semi-dynamic if environment does not change but agent's performance score does
  - Discrete (v/s Continuous): Limited number of distinct, clearly defined percepts and actions
  - Tic-Tac-Toe: Fully Observable, Deterministic, Strategic, Static, Sequential, Multi-agent

Intelligent Agents
- Agent program implements agent function that maps from percept histories to actions
- Agent structures in increasing complexity:
  - Simple reflex agent: Choose action based on current world state
  - Goal-based agent: Keeps track of state for decision making to reach goal
  - Utility-based agent: Chooses action which maximises utility
  - Learning agent: Critiques performance standard, changes decision making to learn
  - Others: Model-based reflex agent, or combinations
- Exploitation v/s Exploration: Balance between learning more about world and maximising gain based on current knowledge

Designing an Agent
- Search Problem: Goal is to find a state or path to a state from a set of possible states by exploring various possibilities
- Agent needs a goal, model of environment, and search algorithm
- Problem Formulation: States, Initial State, Goal / Test State, Actions, Transition Model, Action Cost Function
- Representation Invariant: Ensure that abstract states have corresponding concrete states with restrictions
- Every state has a set of actions, with branching factor b if number of actions is less than b
- Transition Model: Uses transition function to produce next state given state and action

Searching
- Path Cost: Cost of a path from any state to any state
- Optimal Path Cost: Cost of lowest cost path from any state to any state
- Complexity: Time (Number of nodes generated or expanded) or Space (Maximum number of nodes in memory)
- Completeness: Algorithm will find a solution for every problem instance (if one exists)
- Optimal: When an algorithm outputs a solution, it is guaranteed to be optimal (the best possible)

Uninformed Search
BFS
- Maintain a frontier queue, and repeatedly expand shallowest unexpanded node
- Time and Space Complexity: Exponential wrt depth of optimal solution
- Complete if graph is finite
- Optimal if step cost is same everywhere

Uniform Cost Search
- BFS but with non-identical edge weights
- Use a priority queue with path cost instead
- Time and Space Complexity: Exponential wrt tier of optimal solution
- Complete if > 0 step cost everywhere and finite total cost
- Optimal if > 0 step cost everywhere

DFS
- Use a stack to maintain nodes, expand deeper nodes first
- Time Complexity: Exponential wrt max depth of tree
- Space Complexity: Polynomial wrt max depth of tree
- Not Complete when depth of search tree is infinite
- Not Optimal when solution is at shallower depth
- Complete with visited memory

Informed Search
- Has some form of information on search e.g. how good a state is
- Heuristic: Estimate of optimal path cost from any state to the goal state, e.g. straight line distance or Manhattan distance
- Best First Search: Use a priority queue, and evaluate in order of evaluation function of a state
- A* Search: Evaluation function $f(n) = g(n) + h(n)$, where g is the cost to reach node n, and h is the estimated cost from n to goal
- Time and Space Complexity: Exponential
- Complete if edge costs are positive and branching factor is finite
- Optimality depends on heuristic
- Admissible Heuristic: For every node, $h(n) \leq h^*(n)$, where $h^*(n)$ is the optimal path cost to the goal state from n; heuristic never overestimates cost to reach goal
- Theorem: If $h(n)$ is admissible, A* without visited memory is optimal
- Consistent Heuristic: For every node, every successor n' generated by action a satisfies $h(n) \leq c(n, a, n') + h(n')$, $h(G) = 0$, essentially triangle inequality
- Consistency implies Admissibility
- Theorem: If $h(n)$ is consistent, A* with visited memory is optimal
- Relaxed Problem: Has fewer restrictions, optimal solution to relaxed problem is an admissible heuristic
- If $h_1(n) \geq h_2(n)$ for all nodes, then h_1 dominates h_2, and is better for search if admissible

Search Strategies
- Some search algorithms might not terminate without visited memory
- Can set a strict limit on resources an algorithm can use
- Gives rise to a search with polynomial memory which is complete and optimal
- Depth Limited Search: Limit search depth, backtrack when limit is hit
- Time Complexity: Exponential wrt depth limit
- Space Complexity: Polynomial wrt depth limit if used with DFS
- Not complete or optimal (if used with DFS)
- Iterative Deepening Search: Search with increasing depth limits, returning a solution if found
- Time Complexity: Exponential wrt depth of optimal solution
- Space Complexity: Polynomial if used with DFS
- Complete and optimal (if step cost is same everywhere)

Local Search
- Traverse state space by considering only locally-reachable states, repeatedly moving to better ones
- Typically incomplete and suboptimal
- Any-time property: Longer runtimes often have better solutions
- States represent potential solutions which might not be actual states
- Goal Test checks if a current state is the desired solution
- Successor Function generates neighboring states by modifying the current state
- Evaluation Function: Mathematical Function used to assess the quality or desirability of a state which we either want to minimise or maximise
- Hill Climbing: Repeatedly move to the best successor until the current state is the best

Adversarial Search
- Multi-agent problems involve multiple agents which could be either cooperative or competitive
- Adversarial Games: Agents compete against each other; Zero-Sum: One player's gain is the other player's loss
- Turn-Taking Games: Next state of environment also depends on action of strategic opponents
- Game Tree represents all possible states and moves, and outcome is displayed below each terminal node
- Utility Function: Outputs value of a state from the perspective of our agent, which wants to maximise it
- Utility of non-terminal states is typically not known in advance and must be computed

Minimax
- Assumes all players play optimally
- Player A: max, Player B: min
- Time Complexity: Exponential wrt maximum depth
- Space Complexity: Polynomial
- Complete if tree is finite
- Optimal if against optimal opponent
- Theorem: Minimax computes the optimal strategy for both players assuming both play optimally
- Theorem: If the opponent deviates from optimal play, the utility obtained by Player A is never less than the utility they would obtain against an optimal opponent
- Minimax explores the entire game tree in a depth first manner which can be costly
- Alpha-Beta Pruning: A variant of minimax which decreases the number of nodes that need to be evaluated
- α: Highest value for A so far
- β: Lowest value for A so far
- On Player B's turn, if an action leads to a worse outcome, then the rest of the actions are irrelevant since the node will have at most that outcome which Player A does not want
- On Player A's turn, if an action leads to a better outcome, then the rest of the actions are irrelevant since the node will have at least that outcome which Player B is not interested in
- Initialise $\alpha = -\infty, \beta = +\infty$
- At a max node, set v to be $max(v, v_i)$. Then set $\alpha = max(\alpha, v)$. If $\alpha \geq \beta$, prune remaining nodes.
- At a min node, set v to be $min(v, v_i)$. Then set $\beta = min(\beta, v)$. If $\alpha \geq \beta$, prune remaining nodes.
- α, β are not returned by reference!
- Theorem: Alpha-Beta pruning does not alter the minimax value and optimal move computed; it only reduces number of nodes evaluated
- Theorem: With good move ordering, runs in $O(b^{\frac{m}{2}})$
- Cutoff: Apply a strategy to halt search in middle due to computational constraints
- Estimate value of mid-game states with evaluation function which assigns a score to a game state
- A heuristic function estimates utility of a state, which should fall somewhere between the minimum and maximum possible utility for any state
- Derived from relaxed game, expert knowledge, or learnt from data
- Theorem: In a finite two-player, zero-sum game, minimax with cutoff and an evaluation function returns a move that is optimal with respect to the evaluation function at the cutoff (but not necessarily the true value)

```
def max_value(state, a, b):
    v = -INF // +INF
    for next_state in expand(state):
        v = max(v, min_value(next_state, a, b))
```

```
        // min(max)
        a = max(a, v) // b = min(b, v)
        if v >= b: return v // if v <= a:
return v
```

Machine Learning and Decision Trees

Machine Learning

- Some problems are intractable: They do not have an efficient solution for all cases
- Other problems are difficult because formulating the rules in a way that computers can process is hard
- A learning agent learns a function to identify patterns in the data and make decisions rather than explicitly solving it
- Machine Learning is a subfield of AI where computers learn without being explicitly programmed
- Supervised: Learn from labeled data (input-output pairs) to find a mapping from inputs to outputs
- Unsupervised: Learn from unlabeled data (input only) to find patterns or structure
- There is also semi-supervised or reinforcement learning

Supervised Learning

- Data: Set of input-output (ground truth) pairs
- Model: Function that predicts output using input features
- Loss: Function used to assess model's performance
- Learning algorithm seeks to minimise loss of a model over a training dataset
- Model is evaluated on a test set using performance measures to estimate the generalisation of the model
- Classification: Predict a discrete label based on input features; output is a categorical value; Regression: Predict a continuous numerical value based on input features; output is a real number
- A model or hypothesis refers to a function that maps from inputs to outputs and can be learned by a learning algorithm
- Performance measure is a metric evaluating how well the model performs
- A loss function quantifies the difference between the model's predictions and true outputs
- Examples of performance measures include mean squared error or mean absolute error for regression, and accuracy for classification
- For binary classification, we can look at the confusion matrix for true/false positives/negatives: "The prediction is [+/-] and it is [T/F]"
- $Accuracy = \frac{TP+TN}{TP+FN+FP+TN}$
- $Precision = \frac{TP}{TP+FP}$ if false positives are costly
- $Recall = \frac{TP}{TP+FN}$ if false negatives are costly
- $F1 = \frac{2}{1/P+1/R}$
- A learning algorithm takes a training set of input-output pairs and finds a model which approximates the true relationship between inputs and outputs

Decision Trees

- We are given a set of data points consisting of features and a target variable which are categorical
- A decision tree takes in an input vector of categorical attributes and outputs a predicted category
- Prefer more compact decision trees, although they are less likely to be consistent with training data by chance
- At each step, choose the feature that yields the greatest immediate information gain
- Entropy: Measure of uncertainty or randomness in a set of data, or number of bits used to represent it
- Entropy: $H(P(v_1), ...P(v_n)) = -\sum_{i=1}^{n} P(v_i) log_2 P(v_i)$
- Common Entropies: $H(1) = 0, H(1/2, 1/2) = 1, H(1/3, 2/3) = 0.918, H(3/5, 2/5) = 0.971$
- For binary outcomes, $H(\frac{p}{p+n}, \frac{n}{p+n}) = -\frac{p}{p+n} log_2 \frac{p}{p+n} - \frac{n}{p+n} log_2 \frac{n}{p+n}$
- Specific Conditional Entropy: $H(Y|X = x)$ is the entropy of Y conditioned that $X = x$, defined as $H(Y|X = x) = H(P(y_1|x), ..., P(y_k|x))$, only looking at a subset of samples
- Conditional Entropy: $H(Y|X)$ is the entropy of Y when the value of X is known, defined as $H(Y|X) = \sum_{x \in X} P(X = x) H(Y|X = x)$
- Information Gain: Measure of reduction in uncertainty after knowing the value of a specific variable, defined as $IG(Y;X) = H(Y) - H(Y|X)$
- Choose attributes with the highest information gain
- Stop splitting when all rows have same classification
- Theorem (Representational Completeness): For any finite set of consistent examples with discrete features and a finite set of label classes, there exists a decision tree that is consistent with the examples
- However, noise or outliers might cause learned decision tree to be unnecessarily complex or overfitting
- Pruning can produce simple trees e.g. Max depth, Min Sample Leaves
- Partition continuous valued attributes into discrete intervals
- For missing values, come up with a way to populate them

Linear Regression

- Suppose each data point consists of features and a target variable, all of which are real numbers
- Given another data point without a target, we want to find a function that can predict a target for this new data point
- Linear Model: $h_w(x) = w_0x_0 + w_1x_1 + ... + w_dx_d$ where w_i are weights and $x_0 = 1$ is a dummy variable
- $h_w(x) = w^Tx = \sum_{j=0}^{d} w_jx_j$
- We want the linear model with the lowest loss on data, with a common measure being MSE
- MSE: $J_{MSE}(w) = \frac{1}{2n}\sum_{i=1}^{n}(h_w(x^{(i)}) - y^{(i)})^2$
- The gradient of h is $\triangledown h(w) = [\frac{\delta h(w)}{\delta w_1} ... \frac{\delta h(w)}{\delta w_d}]^T = [x_1 ... x_d]^T$
- To minimise a function $f(w)$, we find the point where $\triangledown f(w) = 0$, so for our multidimensional function, we want $\frac{\delta f(w)}{\delta w_i} = 0$ for all w_i
- Minimum loss: $\triangledown J_{MSE}(w) = 0$
- Normal Equation: Use $w = (X^TX)^{-1}X^TY$, derived from $X^T(Xw - Y) = 0$
- However, the cost is $O(d^3)$ for inversion, and requires $(X^TX)^{-1}$ to be invertible
- Gradient Descent: Start at a random w, and update w with a step in opposite direction of gradient repeatedly until some termination criterion is satisfied
- $w_j = w_j - \gamma \frac{\delta J(w_0,...)}{\delta w_j}$ where $\gamma > 0$ is the learning rate
- Convexity: For a line segment connecting any two distinct points on the graph, the line lies on or above the graph between two points
- Theorem: The MSE loss function is convex
- Linearly Dependent Features: A feature which can be expressed as a linear combination of other features
- Theorem: If the features in the training data are linearly independent, the function is strictly convex
- Theorem: For a linear regression model with MSE, gradient descent will converge to a global minimum provided the learning rate is chosen appropriately, which is unique if the features are linearly independent
- Larger features have greater updates to weights, so we need to scale them
- Min-max Scaling: $x_i' = \frac{x_i - min(x_i)}{max(x_i) - min(x_i)}$
- Standardisation: $x_i' = \frac{x_i - \mu_i}{\sigma_i}$, using normal distribution
- Alternatively have different learning rate for each weight
- Gradient descent can be very slow since it requires the entire dataset; can get stuck at local minimum or plateau on non-convex optimisation
- Batch Gradient Descent: Consider all training examples
- Mini-Batch Gradient Descent: Consider a subset of training examples at a time, cheaper per iteration, noisy gradient can help escape local minima
- Stochastic Gradient Descent: Select a random training example at a time
- n points, needs maximum degree of $n - 1$

Feature Transformation

- Transform d dimensional feature into M dimensional, usually $M \geq d$ using a function e.g. polynomial, log, exp
- Theorem: The convexity of linear regression with MSE is preserved with feature transformation
- Model is usually $\hat{h}(x) = h(\phi(x))$, where ϕ is the feature transformation function and h is the predictor

Logistic Regression

- Given a set of data points with features and a target variable, but this time they are described by real numbers, and the target is either a positive or negative class
- This is a classification problem (binary in this case)
- Sigmoid Function: $\sigma(x) = \frac{1}{1+e^{-x}}$, which is differentiable with derivative $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- Logistic Model: Function of the form $h_w(x) = \sigma(w_0x_0 + ... + w_dx_d) = \sigma(w^Tx)$ where $x_0 = 1$ is a dummy variable
- The model's output is the probability of an input to be class 1, giving the confidence score of the model
- Example: Outputs $h_w(x) = P(Fail|x) = 0.8$ means it predicts the probability of failure to be 0.8
- To decide which class an input belongs to, compare the probability output with a given threshold
- Decision Boundary: Surface that separates different classes in the feature space, which is intersection between model's function and decision threshold
- Theorem: MSE is non-convex for logistic regression
- Cross-Entropy: Measures difference between two probability distributions over same set of outcomes, suppose P is the true probability distribution and Q is our model, then we have $H(P,Q) = -\sum_x P(x) log\, Q(x)$
- Binary Cross Entropy: Special case where there are only two possible outcomes, so $H(P,Q) = -P(1) log\, Q(1) - P(0) log\, Q(0)$
- BCE Loss: $J_{BCE}(w) = \frac{1}{n}\sum_{i=1}^{n} BCE(y^{(i)}, h_w(x^{(i)}))$
- Theorem: The BCE loss function is convex
- Partial derivative equation does not have a closed form
- Theorem: For a logistic model with BCE, gradient descent is guaranteed to converge to a global minimum provided the learning rate is chosen appropriately
- Data is linearly separable when a single linear decision boundary separates the different classes
- When it is not, the decision boundary required needs to be non-linear or more complex, which can be done with feature transformation
- Theorem: The convexity of a logistic model with BCE is preserved with feature transformation
- Multi-Class Classification: Either one vs one or one vs rest
- One vs One: Classifier for every pair of classes, need $K(K-1)/2$ of them, class with most votes is selected
- One vs Rest: Treat all other classes as single combined class, need K of them, classifier with highest probability determines the class
- Multi-Label Classification: Labels might no be mutually exclusive
- Binary Relevance: Train one binary classifier per label
- Classifier Chain: Feed predictions of previous labels as features for subsequent labels which can capture dependencies although dependent on order
- Label Powerset: Treat each unique set of labels as a single multiclass label

Topics in Supervised learning

- Generalisation: Model's ability to perform on unseen data
- Dataset Quality: Data should be relevant and balanced, but might contain noise
- Having more data typically leads to better performance
- Model Complexity: Size and expressiveness of model functions; higher model complexity allows for more sophisticated modeling of relationships
- A simple model is good for a simple ground truth but bad for complex ground truth (high bias, underfitting), retraining often leads to same model (low variance)
- Complex model overfits with scarce data, while it can fit better when data is abundant (low bias), retraining can lead to vastly different models (high variance)
- High Complexity: As high error on unseen data, which reduces and converges as data quantity exists
- Low Complexity: High error on both unseen and training data even as data quantity increases
- As model complexity increases, error decreases on unseen then increases again followed by decreasing again due to overparameterisation
- Hyperparameters: Settings that control behaviour of training algorithm but are not learned e.g. learning rate, feature transformations
- Hyperparameter Tuning: Optimising hyperparameters of a machine learning model to improve its performance (using validation set for evaluation)
- Techniques: Grid search, Random search, Local search, Successive halving, Bayesian optimisation